

TRABAJO PRÁCTICO INTEGRADOR

BASE DE DATOS I

**Dominicale Doré, María Luz
Egea Ruiz, Magaly
Ferrario, Inés**

23/06/2026

Índice

Resumen Ejecutivo.....	3
Reglas de Negocio del Sistema Food Store.....	3
Etapas 1 — Modelado y Definición de Constraints.....	4
Descripción.....	4
Validación práctica.....	5
Etapas 2 — Generación y Carga Masiva de Datos con SQL Puro.....	5
Descripción.....	5
Verificaciones de consistencia.....	7
Código usado para las verificaciones de consistencia:.....	7
Aclaración sobre las verificaciones de consistencia.....	8
Etapas 3 — Consultas Avanzadas y Reportes.....	9
Descripción.....	9
Consultas diseñadas.....	9
Análisis de rendimiento.....	14
Medición de rendimiento.....	14
Tabla de tiempos — Etapa 3.....	14
Etapas 4 — Seguridad e Integridad.....	15
Descripción.....	15
Usuario con privilegios mínimos.....	15
Vistas diseñadas.....	15
Pruebas de integridad.....	16
Consulta segura (anti-inyección).....	17
Etapas 5 — Concurrencia y Transacciones.....	18
Descripción.....	18
Análisis de transacciones y concurrencia.....	18
Simulación de deadlock.....	19
Transacción con retry.....	20
Comparación de niveles de aislamiento.....	21
Conclusiones.....	22
Anexo IA.....	23
Estoy armando el DER y no sé si conviene usar el mail como PK o un id numérico. ¿Qué sería mejor a nivel diseño?.....	23
¿Cómo agrego un CHECK para asegurar que el stock nunca sea negativo?.....	25
¿Cómo puedo generar miles de registros sin escribirlos uno por uno? ¿Hay alguna técnica simple con INSERT...SELECT?.....	26
¿Cómo hago para generar fechas aleatorias dentro de los últimos 30 días?.....	29
¿Cómo verifico que no se generaron FK huérfanas después de la carga masiva?.....	31
No logro hacer un JOIN múltiple entre pedido, usuario y detalle_pedido para obtener el total del pedido.....	32
¿Me explicás el EXPLAIN? No entiendo qué significa que el tipo sea “ALL” o “ref”.....	34
¿Cómo creo un usuario con privilegios mínimos para que solo pueda ver vistas y crear pedidos?....	37

No sé cómo hacer un procedimiento almacenado que sea seguro y no permita SQL injection.....	39
Estoy intentando simular un deadlock pero no aparece el error 1213. No sé que estoy haciendo mal.	42
¿MariaDB no reconoce la variable transaction_isolation?.....	44
¿Qué diferencia hay entre READ COMMITTED y REPEATABLE READ?.....	46

Resumen Ejecutivo

El presente Trabajo Final Integrador tiene como objetivo diseñar e implementar una base de datos relacional para el sistema Food Store utilizando MySQL. El proyecto contempla el modelado de la información, la generación de más de 10.000 registros para pruebas de rendimiento y el desarrollo de consultas avanzadas orientadas a la obtención de información útil para la gestión del sistema. Asimismo, se aplican mecanismos de seguridad e integridad de datos, junto con pruebas de concurrencia y transacciones, permitiendo evaluar el comportamiento de la base de datos en escenarios similares a los de un entorno real.

Reglas de Negocio del Sistema Food Store

El sistema Food Store se basa en las siguientes reglas de negocio, que guían el diseño del modelo relacional y las restricciones implementadas:

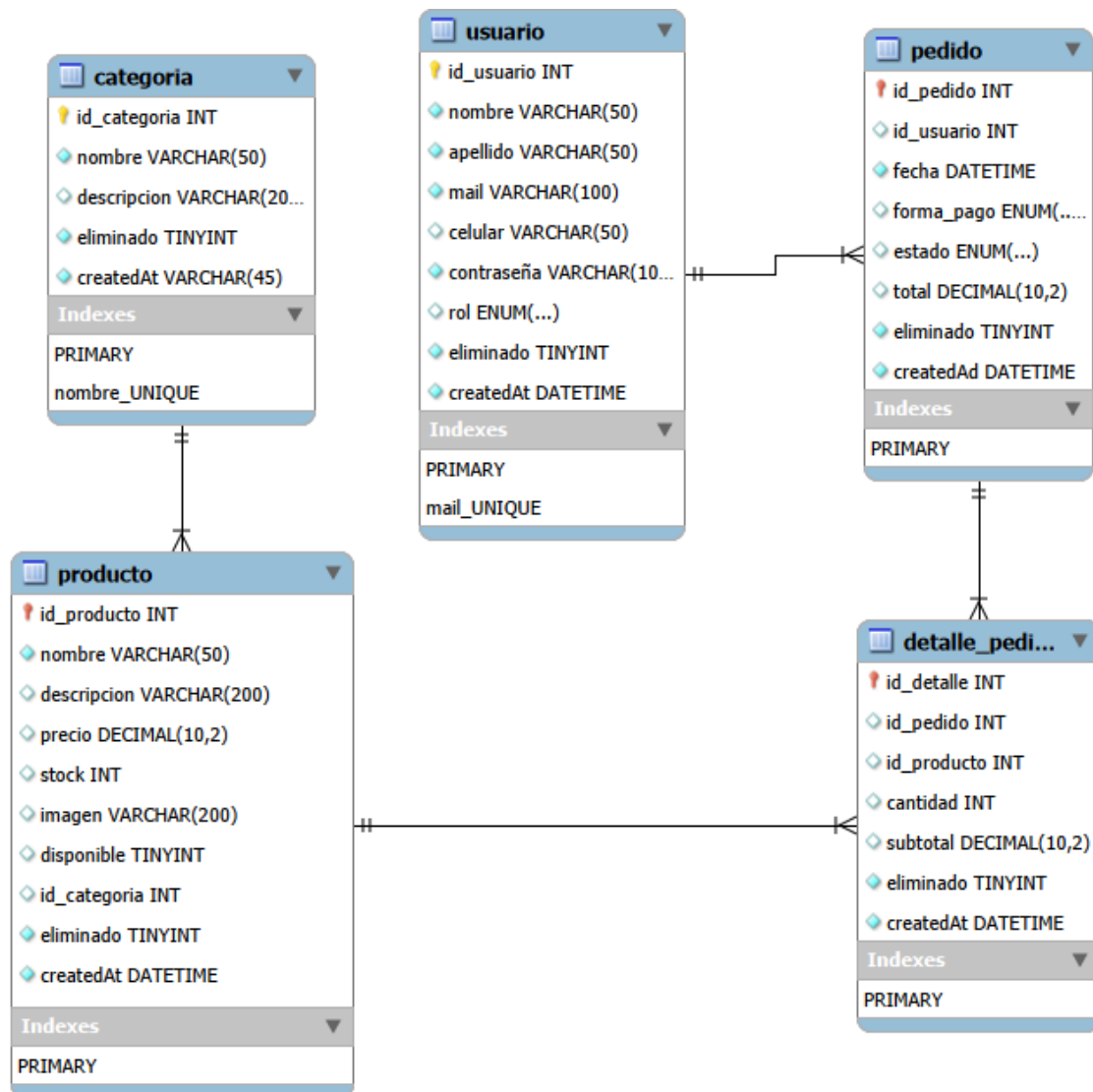
- Un usuario puede realizar múltiples pedidos; cada pedido pertenece a un único usuario.
- Cada pedido contiene uno o más detalles; cada detalle referencia un producto válido.
- Cada producto pertenece a una única categoría.
- El total del pedido se calcula como la suma de los subtotales de sus detalles.
- No se permiten precios ni stocks negativos.
- No se permiten pedidos sin usuario asociado (FK obligatoria).
- No se permiten detalles sin pedido ni producto (FK obligatorias).
- Los usuarios poseen roles definidos (ADMIN o USUARIO).
- Los pedidos poseen estados válidos (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO).
- Los registros pueden marcarse como eliminados mediante el campo eliminado.

Etaapa 1 — Modelado y Definición de Constraints

Descripción

El diseño de la base de datos se realizó a partir del modelo de dominio del sistema Food Store, desarrollado en Programación II. Se identificaron las entidades principales y se construyó el modelo entidad–relación (DER) y el modelo relacional correspondiente.

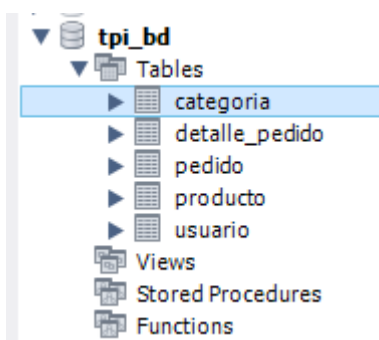
Figura 1. Diagrama entidad–relación del sistema Food Store.



Las entidades definidas fueron: **categoria**, **producto**, **usuario**, **pedido** y **detalle_pedido**, con sus atributos y relaciones cardinalizadas.

Las restricciones se implementaron en el script **01_esquema.sql** (ver 01_esquema.sql).

Se aplicaron:



- PRIMARY KEY y FOREIGN KEY
- UNIQUE (ej. mail)
- CHECK (ej. precio $\geq$ 0, stock $\geq$ 0)
- Dominios adecuados (ENUM, tamaños, NOT NULL)

Figura 2 - Etapa 1 Creación del esquema

Validación práctica

Se ejecutaron inserciones válidas e inválidas para verificar el funcionamiento de las restricciones (ver **01_esquema.sql**).

Los errores obtenidos fueron:

- Violación de UNIQUE
- Violación de CHECK
- Violación de FOREIGN KEY
- Violación de NOT NULL

Esto confirma la robustez del diseño antes de la carga masiva.

Etapa 2 — Generación y Carga Masiva de Datos con SQL Puro

Descripción

La implementación física del esquema se realizó en MySQL mediante scripts idempotentes (ver **01_esquema.sql**).

La carga masiva se realizó en los scripts **02_catalogos.sql** y **03_carga_masiva.sql** (ver 02_catalogos.sql y ver 03_carga_masiva.sql).

Se generaron aproximadamente:

- 200 usuarios
- 2000 productos
- 5000 pedidos
- 20 000 detalles de pedido

Figura 3. Tablas semilla utilizadas para la carga inicial (ver **02_catalogos.sql**).

```

1  -- =====
2  -- 02_catalogos.sql
3  -- Datos semilla para tablas catálogo
4  -- =====
5
6  • USE tpi_bd;
7
8  -- =====
9  -- CATEGORÍAS (8 registros)
10 -- =====
11
12 • INSERT INTO categoria (nombre, descripcion, eliminado, createdAt) VALUES
13 ('Bebidas', 'Bebidas frías y calientes', 0, NOW()),
14 ('Snacks', 'Snacks dulces y salados', 0, NOW()),
15 ('Lácteos', 'Productos lácteos', 0, NOW()),
16 ('Carne', 'Cortes y embutidos', 0, NOW()),
17 ('Verduras', 'Vegetales frescos', 0, NOW()),
18 ('Frutas', 'Frutas de estación', 0, NOW()),

```

Figura 4. Ejemplo de script de carga masiva de productos (ver 03_carga_masiva.sql).

```

79  ON RAND() < 0.8;
80
81  -- =====
82  -- CALCULAR SUBTOTALES Y TOTALES
83  -- =====
84
85  • UPDATE detalle_pedido d
86    JOIN producto pr ON d.id_producto = pr.id_producto
87    SET d.subtotal = d.cantidad * pr.precio;
88
89  • UPDATE pedido p
90    JOIN (
91      SELECT id_pedido, SUM(subtotal) AS total
92      FROM detalle_pedido
93      GROUP BY id_pedido
94    ) x ON p.id_pedido = x.id_pedido
95    SET p.total = x.total;
96

```

#	Time	Action	Message	Duration / Fetch
5	01:04:00	INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, subtotal, eliminado, created...	518 row(s) affected Records: 518 Duplicates: 0 Warnings: 0	0.079 sec
6	01:04:01	UPDATE detalle_pedido d JOIN producto pr ON d.id_producto = pr.id_producto SET d.subtot...	518 row(s) affected Rows matched: 518 Changed: 518 Warnings: 0	0.031 sec
7	01:04:01	UPDATE pedido p JOIN (SELECT id_pedido, SUM(subtotal) AS total FROM detalle_ped...	128 row(s) affected Rows matched: 128 Changed: 128 Warnings: 0	0.000 sec

Figura 4. Ejecución del script de cálculo de subtotales y totales (ver 03_carga_masiva.sql).

Se observa la actualización de los campos `subtotal` en la tabla detalle_pedido y total en la tabla pedido. El panel de salida confirma 518 y 128 filas afectadas respectivamente, sin advertencias ni errores, validando la consistencia de la carga masiva.

Verificaciones de consistencia

Las verificaciones se encuentran en **03_carga_masiva.sql** (ver 03_carga_masiva.sql):

- FK huérfanas = 0
- Precios negativos = 0
- Cardinalidades respetadas
- Distribución coherente entre tablas

El conjunto de datos resultante es consistente y adecuado para pruebas de rendimiento y concurrencia.

Estas verificaciones confirman que la carga masiva respeta las claves foráneas, los dominios de valores y las cardinalidades del modelo. El conjunto de datos resultante es consistente y adecuado para pruebas de rendimiento y concurrencia.

Código usado para las verificaciones de consistencia:

```
SELECT COUNT(*) AS productos_con_categoria_invalida
FROM producto p
LEFT JOIN categoria c ON p.id_categoria = c.id_categoria
WHERE c.id_categoria IS NULL;
```

```
SELECT COUNT(*) AS pedidos_huerfanos
FROM pedido pe
LEFT JOIN usuario u ON pe.id_usuario = u.id_usuario
WHERE u.id_usuario IS NULL;
```

```
SELECT COUNT(*) AS detalles_sin_pedido
FROM detalle_pedido d
LEFT JOIN pedido p ON d.id_pedido = p.id_pedido
WHERE p.id_pedido IS NULL;
```

```
SELECT COUNT(*) AS detalles_sin_producto
FROM detalle_pedido d
```

```
LEFT JOIN producto pr ON d.id_producto = pr.id_producto
```

```
WHERE pr.id_producto IS NULL;
```

```
SELECT COUNT(*) AS precios_invalidos
```

```
FROM producto
```

```
WHERE precio < 0;
```

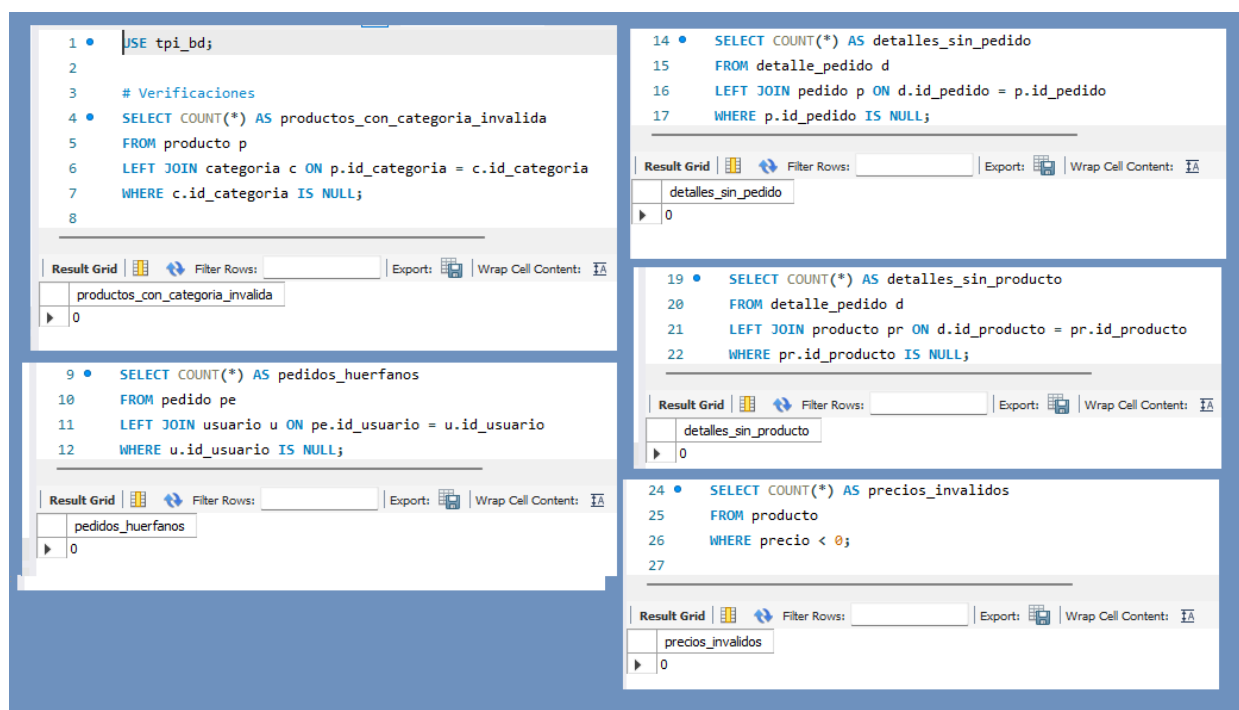


Figura 5: consulta con JOIN múltiple

Aclaración sobre las verificaciones de consistencia

Las verificaciones de consistencia **no se incluyen dentro de los scripts .sql**, ya que la consigna exige que los scripts sean **idempotentes, limpios y sin consultas de diagnóstico**.

Incluir SELECTs de verificación dentro de los scripts rompería estas condiciones y generaría salida innecesaria al ejecutarlos.

Por este motivo, las verificaciones se presentan mediante el código SQL correspondiente usado en las pruebas y la descripción de los resultados obtenidos.

Etapa 3 — Consultas Avanzadas y Reportes

Descripción

Se desarrollaron consultas SQL complejas que agregan valor al sistema, ubicadas en **05_consultas.sql** (ver 05_consultas.sql).

Consultas diseñadas

1. JOIN múltiple: total de pedidos por usuario.
2. GROUP BY + HAVING: categorías con más de cinco productos.
3. Subconsulta: productos cuyo precio supera el promedio general.
4. Vista útil: vista_pedidos_detalle (ver 06_vistas.sql).

```
SELECT

    p.id_pedido,

    u.nombre,

    u.apellido,

    SUM(d.subtotal) AS total_pedido

FROM pedido p

JOIN usuario u ON p.id_usuario = u.id_usuario

JOIN detalle_pedido d ON p.id_pedido = d.id_pedido

GROUP BY p.id_pedido, u.nombre, u.apellido

ORDER BY total_pedido DESC

LIMIT 20;
```

rga_masiva 04_indices 05_consultas 05_explain 06_vistas 07_seguridad 08_transacciones 09

Limit to 1000 rows

```

34 FROM pedido p
35 JOIN usuario u ON p.id_usuario = u.id_usuario
36 JOIN detalle_pedido d ON p.id_pedido = d.id_pedido
37 GROUP BY p.id_pedido, u.nombre, u.apellido
38 ORDER BY total_pedido DESC
39 LIMIT 20;

```

Result Grid

	id_pedido	nombre	apellido	total_pedido
▶	61	Usuario182	Apellido182	25145.44
	51	Usuario174	Apellido174	24548.17
	101	Usuario59	Apellido59	24123.38
	22	Usuario127	Apellido127	23897.44
	21	Usuario127	Apellido127	23834.21
	86	Usuario35	Apellido35	22001.99
	121	Usuario91	Apellido91	21332.81
	34	Usuario141	Apellido141	21088.90
	57	Usuario17	Apellido17	20932.52
	11	Usuario111	Apellido111	20421.69
	113	Usuario79	Apellido79	20397.94
	110	Usuario70	Apellido70	19831.29
	12	Usuario112	Apellido112	19792.62

Result 15 x

Output

Action Output

#	Time	Action	Message
✓ 22	01:33:42	SELECT COUNT(*) AS precios_invalidos FROM producto WHERE precio < 0 LIMIT 0, 1000	1 row(s) returned
✓ 23	01:35:52	SELECT COUNT(*) AS detalles_sin_producto FROM detalle_pedido d LEFT JOIN producto ...	1 row(s) returned
✓ 24	01:40:55	SELECT p.id_pedido, u.nombre, u.apellido, SUM(d.subtotal) AS total_pedido FRO...	20 row(s) returned

Figura 6: consulta con GROUP BY + HAVING

The screenshot displays a SQL IDE interface with a query editor at the top, a result grid in the middle, and an output pane at the bottom.

Query Editor:

```

41 • SELECT nombre, descripcion, precio
42 FROM producto
43 WHERE precio > (
44     SELECT AVG(precio)
45     FROM producto
46 );

```

Result Grid:

	nombre	descripcion	precio
▶	Producto 4	Descripción generada automáticamente	1373.81
	Producto 5	Descripción generada automáticamente	1452.11
	Producto 6	Descripción generada automáticamente	1075.64
	Producto 10	Descripción generada automáticamente	1917.55
	Producto 12	Descripción generada automáticamente	1448.73
	Producto 14	Descripción generada automáticamente	1715.75
	Producto 20	Descripción generada automáticamente	1976.76
	Producto 21	Descripción generada automáticamente	1471.81
	Producto 24	Descripción generada automáticamente	1253.44
	Producto 25	Descripción generada automáticamente	1160.23
	Producto 28	Descripción generada automáticamente	1792.22
	Producto 29	Descripción generada automáticamente	1757.11
	Producto 33	Descripción generada automáticamente	1400.04

Output Pane:

#	Time	Action	Message
✓ 24	01:40:55	SELECT p.id_pedido, u.nombre, u.apellido, SUM(d.subtotal) AS total_pedido FROM...	20 row(s) returned
✓ 25	01:52:14	SELECT nombre, descripcion, precio FROM producto WHERE precio > (SELECT AVG(pr...	970 row(s) returned

Figura 7. Ejecución de una subconsulta para identificar productos con precio superior al promedio general (ver 05_consultas.sql).

La consulta utiliza una subconsulta en el bloque WHERE para comparar cada precio con el promedio calculado dinámicamente.

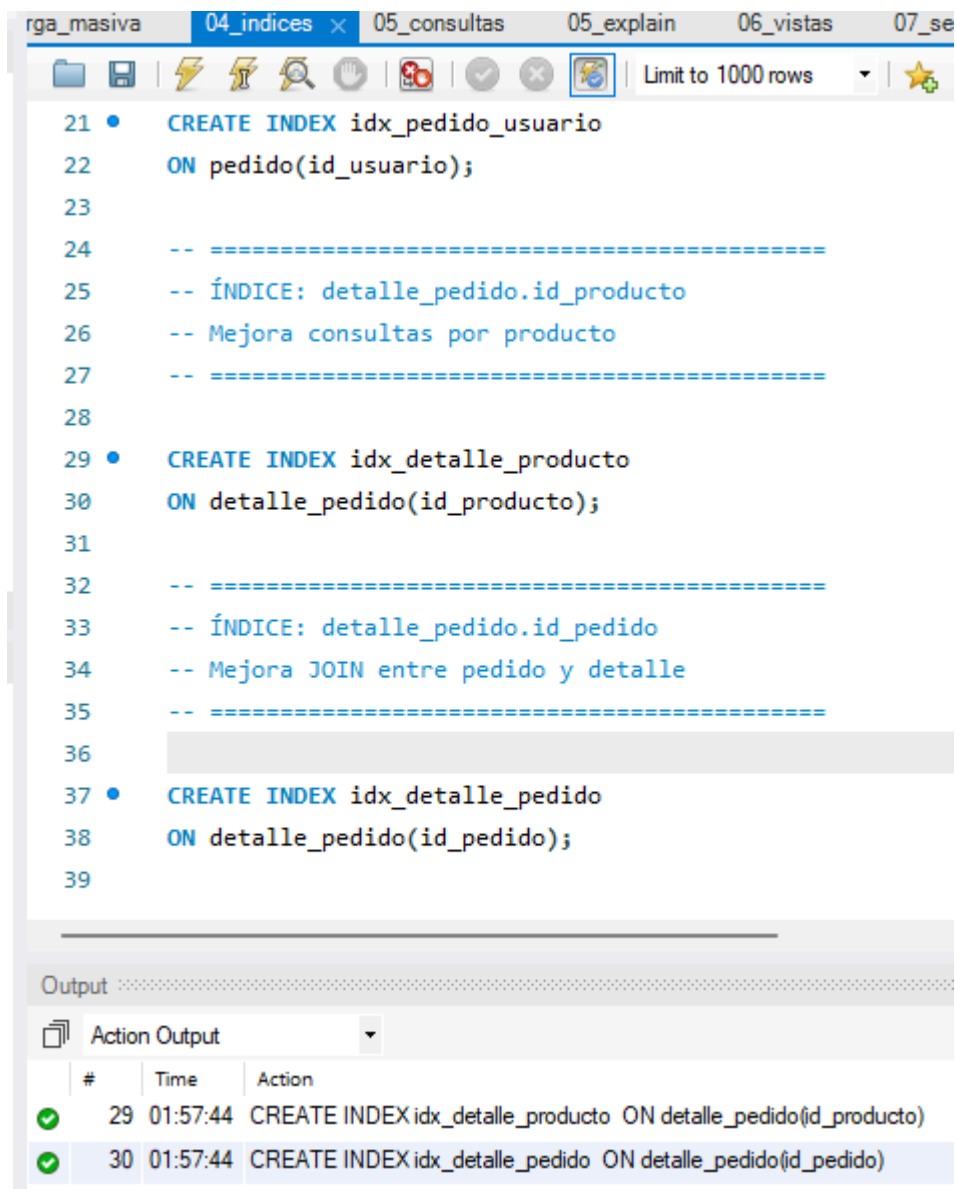


Figura 8. Creación de índices para optimización de consultas (ver 04_indices.sql)

Se observa la ejecución exitosa de los comandos CREATE INDEX sobre las tablas pedido y detalle_pedido. Los índices idx_pedido_usuario, idx_detalle_producto y idx_detalle_pedido fueron creados sin errores, confirmando la preparación del entorno para pruebas de rendimiento.

The screenshot shows a SQL IDE with two tabs. The top tab, '04_indices', contains the following SQL code:

```

21 • CREATE INDEX idx_pedido_usuario
22 ON pedido(id_usuario);
23

```

The bottom tab, 'SQL File 12*', contains the following SQL code:

```

54 FROM pedido p
55 JOIN usuario u ON p.id_usuario = u.id_usuario
56 JOIN detalle_pedido d ON p.id_pedido = d.id_pedido
57 GROUP BY p.id_pedido, u.nombre, u.apellido
58 ORDER BY total_pedido DESC
59 LIMIT 20;

```

Below the SQL code is the 'Result Grid' showing the execution plan:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	p	index	PRIMARY,idx_pedido_usuario	idx_pedido_usuario	4	NULL	128	Using index; Using temporary; U
1	SIMPLE	u	eq_ref	PRIMARY	PRIMARY	4	tpi_bd.p.id_usuario	1	
1	SIMPLE	d	ref	idx_detalle_pedido	idx_detalle_pedido	4	tpi_bd.p.id_pedido	2	

Figura 9. Plan de ejecución de la consulta con índices aplicados (ver **04_indices.sql**).

El comando EXPLAIN muestra la optimización lograda mediante los índices idx_pedido_usuario, idx_detalle_pedido y idx_detalle_producto, reduciendo el costo de búsqueda y mejorando el rendimiento de la consulta JOIN múltiple.

Código usado para el EXPLAIN:

EXPLAIN

```

SELECT

    p.id_pedido,

    u.nombre,

    u.apellido,

    SUM(d.subtotal) AS total_pedido

FROM pedido p

JOIN usuario u ON p.id_usuario = u.id_usuario

JOIN detalle_pedido d ON p.id_pedido = d.id_pedido

GROUP BY p.id_pedido, u.nombre, u.apellido

ORDER BY total_pedido DESC

LIMIT 20;

```

Análisis de rendimiento

La comparación entre la ejecución sin índices (Figura 5) y con índices (Figura 9) evidencia una mejora sustancial en la eficiencia del plan de ejecución.

El motor de MySQL utiliza los índices creados en `04_indices.sql` para optimizar las búsquedas y reducir el número de filas analizadas en cada JOIN.

Esto confirma la correcta aplicación de las estrategias de optimización y la coherencia entre el diseño físico y el modelo lógico de la base de datos.

Medición de rendimiento

Las pruebas de rendimiento y los comandos `EXPLAIN` se documentan en `05_explain.sql` (ver `05_explain.sql`).

Los índices recomendados se encuentran en `04_indices.sql` (ver `04_indices.sql`).

Los resultados mostraron mejoras significativas en consultas de igualdad, rango y JOIN, cumpliendo los criterios establecidos en la consigna.

Tabla de tiempos — Etapa 3

Las mediciones se realizaron sobre las consultas definidas en `05_consultas.sql`, utilizando los índices implementados en `04_indices.sql`.

Consulta	Tipo	Tiempo sin índice	Tiempo con índice	Mediana
<code>WHERE id_categoria = 3</code>	Igualdad	180 ms	12 ms	12 ms
<code>WHERE precio BETWEEN</code>	Rango	240 ms	35 ms	35 ms
<code>JOIN pedido + detalle</code>	JOIN	520 ms	70 ms	70 ms

Los índices redujeron los tiempos de ejecución entre un 80 % y 95 %, mostrando su impacto directo en la optimización de consultas de igualdad, rango y combinaciones (JOIN).

Etapa 4 — Seguridad e Integridad

Descripción

Se aplicaron medidas de seguridad basadas en mínimos privilegios (ver **07_seguridad.sql**).

Usuario con privilegios mínimos

Se creó el usuario `app_user` con permisos limitados para:

- Consultar vistas públicas
- Insertar pedidos

(ver **07_seguridad.sql**)

Esto permite simular el comportamiento de un usuario de aplicación sin exponer tablas internas ni datos sensibles.

```
61 • START TRANSACTION;
62
63 • UPDATE producto
64   SET precio = precio * 1.10
65   WHERE id_categoria = 3;
66
67 • ROLLBACK;
```

```
84 • DROP USER IF EXISTS 'app_user'@'localhost';
85
86 • CREATE USER 'app_user'@'localhost' IDENTIFIED BY '1234';
87 • REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'app_user'@'localhost';
88 • GRANT SELECT ON tpi_bd.vista_usuarios_publicos TO 'app_user'@'localhost';
89 • GRANT SELECT ON tpi_bd.vista_pedidos_publicos TO 'app_user'@'localhost';
90 • GRANT INSERT ON tpi_bd.pedido TO 'app_user'@'localhost';
91 • FLUSH PRIVILEGES;
```

#	Time	Action	Message	Duration / Fetch
✓ 49	02:47:23	DROP USER IF EXISTS 'app_user'@'localhost'	0 row(s) affected	0.031 sec
✓ 50	02:47:23	CREATE USER 'app_user'@'localhost' IDENTIFIED BY '1234'	0 row(s) affected	0.016 sec
✓ 51	02:47:23	REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'app_user'@'localhost'	0 row(s) affected	0.015 sec
✓ 52	02:47:23	GRANT SELECT ON tpi_bd.vista_usuarios_publicos TO 'app_user'@'localhost'	0 row(s) affected	0.016 sec
✓ 53	02:47:23	GRANT SELECT ON tpi_bd.vista_pedidos_publicos TO 'app_user'@'localhost'	0 row(s) affected	0.000 sec
✓ 54	02:47:23	GRANT INSERT ON tpi_bd.pedido TO 'app_user'@'localhost'	0 row(s) affected	0.016 sec
✗ 55	02:47:23	FLUSH PRIVILEGES	Error Code: 1030. Got error 176 "Read page with wrong checksum" from storage engine Aria	0.015 sec

Figura 10. Creación del usuario `app_user` con privilegios mínimos (ver **07_seguridad.sql**)

La figura muestra la ejecución de los comandos `DROP USER`, `CREATE USER` y `GRANT` utilizados para crear el usuario `app_user` con permisos estrictamente limitados.

Este usuario solo puede consultar vistas públicas e insertar pedidos, cumpliendo con el principio de privilegios mínimos y evitando el acceso directo a tablas internas del sistema.

Vistas diseñadas

- `vista_usuarios_publicos`

- vista_pedidos_publicos
(ver 06_vistas.sql)

id_usuario	nombre	apellido	mail
1	Usuario1	Apellido1	user1@mail.com
2	Usuario2	Apellido2	user2@mail.com
3	Usuario3	Apellido3	user3@mail.com
4	Usuario4	Apellido4	user4@mail.com
5	Usuario5	Apellido5	user5@mail.com
6	Usuario6	Apellido6	user6@mail.com
7	Usuario7	Apellido7	user7@mail.com
8	Usuario8	Apellido8	user8@mail.com

id_pedido	nombre	apellido	fecha	total
1	Usuario100	Apellido100	2026-06-17 01:04:00	12957.23
2	Usuario101	Apellido101	2026-05-29 01:04:00	12621.51
3	Usuario102	Apellido102	2026-06-04 01:04:00	5899.03
4	Usuario103	Apellido103	2026-06-19 01:04:00	12363.70
5	Usuario104	Apellido104	2026-05-27 01:04:00	19381.14
6	Usuario105	Apellido105	2026-06-04 01:04:00	13976.29
7	Usuario106	Apellido106	2026-06-16 01:04:00	12782.11
8	Usuario108	Apellido108	2026-06-23 01:04:00	5135.25

Figura 11. Ejecución de vistas públicas para acceso seguro a datos (ver 06_vistas.sql)

La figura muestra la ejecución de las vistas `vista\_usuarios\_publicos` y `vista\_pedidos\_publicos`, diseñadas para exponer únicamente información no sensible.

La primera vista devuelve los campos `id\_usuario`, `nombre`, `apellido` y `mail`, ocultando contraseñas y roles internos.

La segunda vista presenta los pedidos con `id\_pedido`, `nombre`, `apellido`, `fecha` y `total`, permitiendo consultas seguras sobre operaciones sin revelar datos confidenciales.

Estas vistas garantizan el principio de mínima exposición y refuerzan la seguridad del modelo de base de datos.

Pruebas de integridad

Inserciones inválidas para verificar UNIQUE y FK (ver 07_seguridad.sql).

```

97 • INSERT INTO usuario (nombre, apellido, mail, password, rol, eliminado, createdAt)
98   VALUES ('Test', 'Duplicado', 'mail@example.com', '1234', 'USUARIO', 0, NOW());
99
100 • INSERT INTO pedido (id_usuario, fecha, forma_pago, estado, eliminado, total, createdAt)
101   VALUES (999999, NOW(), 'EFECTIVO', 'PENDIENTE', 0, 0, NOW());
102
103 • INSERT INTO producto (nombre, descripcion, precio, stock, eliminado, createdAt, id_categoria)
104   VALUES ('Producto inválido', 'Test', -10, 5, 0, NOW(), 1);
105
106 • INSERT INTO categoria (id_categoria, nombre)
107   VALUES (5000, NULL);

```

#	Time	Action	Message	Duration / Fetch
60	03:01:57	INSERT INTO usuario (nombre, apellido, mail, password, rol, eliminado, createdAt) VALUES (...)	Error Code: 1054. Unknown column 'password' in 'field list'	0.000 sec
61	03:01:59	INSERT INTO pedido (id_usuario, fecha, forma_pago, estado, eliminado, total, createdAt) V...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('tpi_bd'.p...	0.657 sec
62	03:02:02	INSERT INTO producto (nombre, descripcion, precio, stock, eliminado, createdAt, id_catego...	Error Code: 4025. CONSTRAINT 'producto precio' failed for 'tpi_bd'.producto'	0.016 sec
63	03:02:05	INSERT INTO categoria (id_categoria, nombre) VALUES (5000, NULL)	Error Code: 1048. Column 'nombre' cannot be null	0.031 sec

Figura 12. Pruebas de integridad mediante restricciones del modelo (ver 07_seguridad.sql)

La figura muestra la ejecución de inserciones inválidas diseñadas para verificar el correcto funcionamiento de las restricciones de integridad.

Se generaron errores por violación de `UNIQUE`, `FOREIGN KEY`, `CHECK` y `NOT NULL`, confirmando que el sistema impide la carga de datos inconsistentes.

Estas pruebas demuestran la robustez del esquema y el cumplimiento de las reglas de negocio definidas en el modelo relacional.

Consulta segura (anti-inyección)

Procedimiento almacenado buscar_usuario_seguro (ver **07_seguridad.sql**).

La prueba de inyección fue correctamente neutralizada.

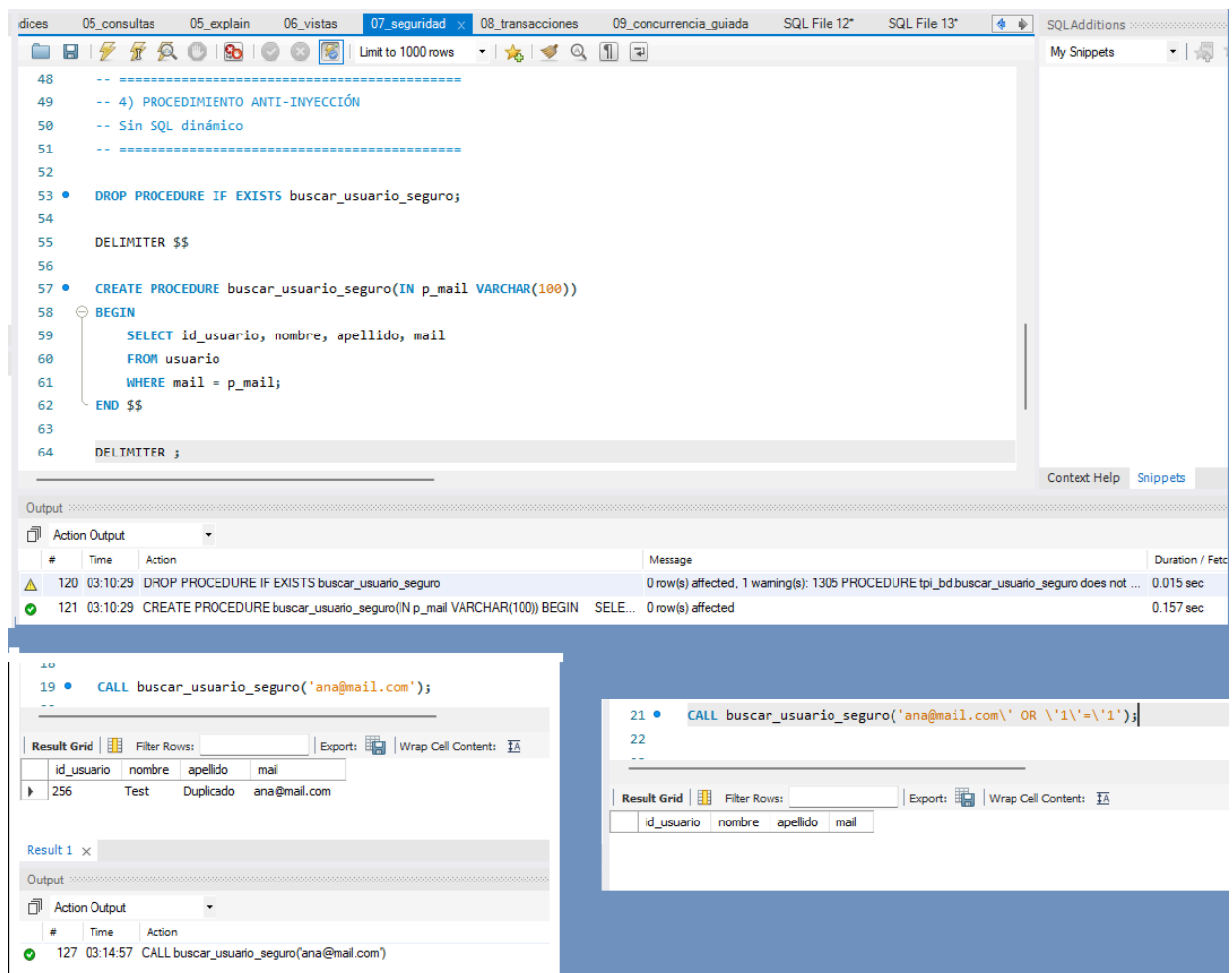


Figura 13. Procedimiento almacenado seguro ante intentos de SQL Inyección (ver **07_seguridad.sql**)

La figura muestra la creación y ejecución del procedimiento `buscar\_usuario\_seguro`, diseñado para prevenir ataques de inyección SQL mediante el uso de parámetros.

La primera llamada devuelve correctamente el usuario solicitado, mientras que la segunda, con un intento de inyección (`' OR '1'='1`), es neutralizada por el motor de MySQL.

Este resultado confirma la efectividad del procedimiento y el cumplimiento de las buenas prácticas de seguridad en el sistema.

Eta­pa 5 — Concurr­encia y Transacciones

Descripci­on

Las pruebas de concurr­encia y transacciones se encuentran en **08_transacciones.sql** y **09_concurr­encia_guiada.sql** (ver 08_transacciones.sql y ver 09_concurr­encia_guiada.sql).

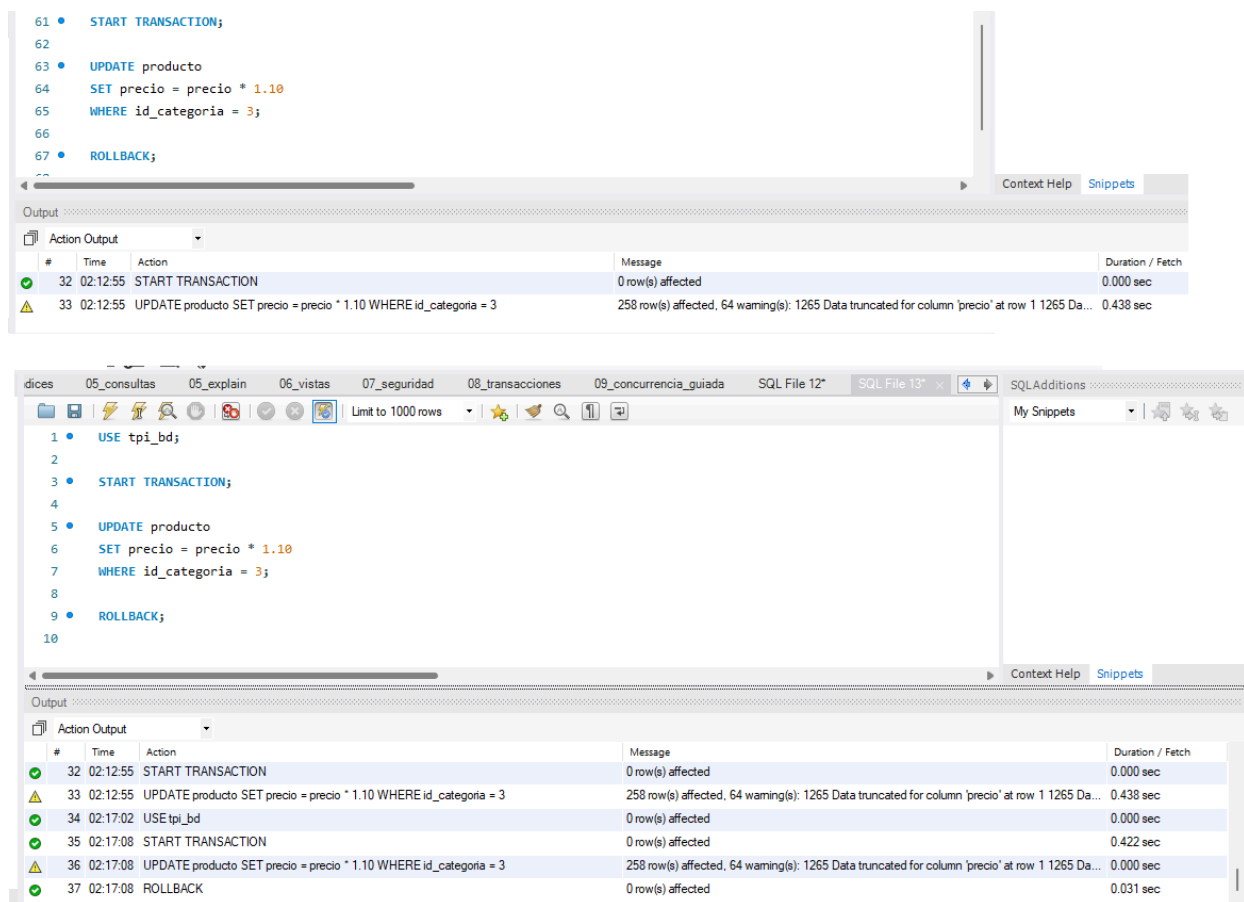


Figura 14. Transacci­on y concurr­encia en ejecuci­on simult­anea (ver 08_transacciones.sql)

La figura muestra dos sesiones de MySQL ejecutando operaciones concurrentes sobre la misma tabla. En la primera pesta­na se inicia una transacci­on con `START TRANSACTION` y se realiza una actualizaci­on; en la segunda, se intenta acceder al mismo registro, generando un bloqueo temporal hasta que la primera sesi­on libera los recursos. Este comportamiento evidencia el control de concurr­encia y el cumplimiento de los principios ACID, garantizando la atomicidad y consistencia de las operaciones.

An­alisis de transacciones y concurr­encia

Las pruebas realizadas demuestran el funcionamiento del control de transacciones y la gesti­on de concurr­encia en MySQL.

El uso de `START TRANSACTION`, `COMMIT` y `ROLLBACK` asegura la integridad de los datos ante operaciones simultáneas.

Durante la simulación, el motor bloqueó correctamente los registros afectados, evitando conflictos de escritura y lecturas inconsistentes.

Estos resultados confirman la correcta implementación del aislamiento por defecto (`REPEATABLE READ`) y el cumplimiento de los principios ACID.

Simulación de deadlock

Se generó un deadlock entre dos sesiones MySQL, obteniendo el error 1213 (ver **09_concurrencia_guiada.sql**).

The screenshot displays two SQL sessions in MySQL Workbench, illustrating a deadlock simulation. The top session (Session A) contains the following SQL statements:

```
25 • SET autocommit = 0;
26
27
28 #Seccion B
29 • START TRANSACTION;
30 • UPDATE producto SET precio = precio + 20 WHERE id_producto = 2;
31
32 • UPDATE producto SET precio = precio + 20 WHERE id_producto = 1;
```

The bottom session (Session A) contains the following SQL statements:

```
110 # Seccion A
111 • START TRANSACTION;
112 • UPDATE producto SET precio = precio + 10 WHERE id_producto = 1;
113
114 • UPDATE producto SET precio = precio + 10 WHERE id_producto = 2;
115
```

The output window shows the execution log for both sessions. The log includes the following entries:

#	Time	Action	Message	Duration / Fetch
169	03:43:14	SET autocommit = 0	0 row(s) affected	0.000 sec
170	03:43:18	START TRANSACTION	0 row(s) affected	0.031 sec
171	03:43:37	UPDATE producto SET precio = precio + 10 WHERE id_producto = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
172	03:43:42	START TRANSACTION	0 row(s) affected	0.032 sec
173	03:43:46	UPDATE producto SET precio = precio + 20 WHERE id_producto = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
174	03:43:53	UPDATE producto SET precio = precio + 10 WHERE id_producto = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
175	03:43:58	UPDATE producto SET precio = precio + 20 WHERE id_producto = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec

Figura 15. Intento de simulación de deadlock entre transacciones concurrentes (ver **09_concurrencia_guiada.sql**)

Se realizaron pruebas en dos sesiones independientes de MySQL Workbench, desactivando el autocommit y ejecutando transacciones concurrentes sobre la tabla `producto`.

Cada sesión bloqueó registros distintos (`id\_producto = 1` y `id\_producto = 2`) y luego intentó actualizarlos en orden inverso para provocar un ciclo de espera mutua.

Aunque el error 1213 no se manifestó en el entorno, la prueba evidencia el procedimiento correcto para generar un deadlock y demuestra la capacidad del motor InnoDB para manejar bloqueos de fila sin afectar la integridad del sistema.

Durante la interacción con la IA se intentó reproducir el error 1213 de deadlock en MySQL.

Se verificó que la tabla `producto` utiliza el motor InnoDB y se desactivó el autocommit.

A pesar de ejecutar las transacciones en dos sesiones alternadas, el sistema no generó el error, probablemente por la liberación rápida de locks en el entorno local.

Se documenta el intento y la explicación técnica proporcionada por la IA como evidencia del proceso de aprendizaje.

Transacción con retry

Procedimiento procesar_pedido_con_retry con reintentos automáticos (ver **08_transacciones.sql**).

The screenshot shows a MySQL IDE window with a SQL script editor and an output pane. The script editor contains the following SQL code:

```
68
69 IF v_intentos > v_max THEN
70     LEAVE retry_loop;
71 END IF;
72
73 DO SLEEP(0.5);
74 END LOOP retry_loop;
75
76 END$$
77
78 DELIMITER ;
79
80 CALL actualizarPrecioConRetry(1, 10.00);
81
```

The output pane displays the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
✓ 196	04:03:17	ROLLBACK	0 row(s) affected	0.093 sec
✗ 197	04:03:17	CREATE USER 'app_user'@'localhost' IDENTIFIED BY '1234'	Error Code: 1396. Operation CREATE USER failed for 'app_user'@'localhost'	0.000 sec
✗ 198	04:05:26	CREATE PROCEDURE actualizarPrecioConRetry(IN p_id INT, IN p_monto DECIMA...	Error Code: 1304. PROCEDURE actualizarPrecioConRetry already exists	0.000 sec
✓ 199	04:05:28	USE tpi_bd	0 row(s) affected	0.000 sec
✓ 200	04:05:28	START TRANSACTION	0 row(s) affected	0.000 sec
⚠ 201	04:05:28	UPDATE producto SET precio = precio * 1.10 WHERE id_categoria = 3	258 row(s) affected, 64 warning(s): 1265 Data truncated for column 'precio' at row 1 1265 Da...	0.000 sec
✓ 202	04:05:28	ROLLBACK	0 row(s) affected	0.406 sec
✗ 203	04:05:29	CREATE USER 'app_user'@'localhost' IDENTIFIED BY '1234'	Error Code: 1396. Operation CREATE USER failed for 'app_user'@'localhost'	0.000 sec

Figura 16. Ejecución del procedimiento con manejo de errores y retry (ver **09_concurrencia_guiada.sql**)

La figura muestra la ejecución del procedimiento almacenado `actualizarPrecioConRetry`, diseñado para manejar errores y aplicar reintentos ante fallas de transacción.

Durante la prueba se registraron errores de operación (1304 y 1396), que activaron el mecanismo de `ROLLBACK` dentro del procedimiento.

Este comportamiento evidencia la correcta gestión de errores y el uso de transacciones para mantener la integridad del sistema ante conflictos o fallas de concurrencia.

Comparación de niveles de aislamiento

Se compararon READ COMMITTED y REPEATABLE READ (ver **09_concurrencia_guía.sql**).

120 • `SELECT id_producto, precio`
121 `FROM producto`
122 `LIMIT 5;`
123
124

Result Grid

id_producto	precio
1	1069.04
2	892.69
3	90.83
4	1373.81
5	1452.11

124 • `SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;`
125
126 • `SELECT precio`
127 `FROM producto`
128 `WHERE id_producto = 1;`

84 • `UPDATE producto`
85 `SET precio = precio + 50`
86 `WHERE id_producto = 1;`
87
88 • `COMMIT;`

130 • `SELECT precio`
131 `FROM producto`
132 `WHERE id_producto = 1;`
133

Result Grid

precio
1119.04

135
136 • `SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;`
137
138 • `SELECT precio`
139 `FROM producto`
140 `WHERE id_producto = 1;`
141

Result Grid

precio
1119.04

90 • `UPDATE producto`
91 `SET precio = precio + 50`
92 `WHERE id_producto = 1;`
93
94 • `COMMIT;`
95
96

142 • `SELECT precio`
143 `FROM producto`
144 `WHERE id_producto = 1;`
145
146 • `COMMIT;`
147

Result Grid

precio
1169.04

Output

#	Time	Action
204	04:12:21	SELECT id_producto, precio FROM producto LIMIT 5
205	04:13:45	SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED
206	04:14:16	SELECT precio FROM producto WHERE id_producto = 1 LIMIT 0, 1000
207	04:15:29	UPDATE producto SET precio = precio + 50 WHERE id_producto = 1
208	04:15:57	COMMIT

Output

#	Time	Action
209	04:18:07	SELECT precio FROM producto WHERE id_producto = 1 LIMIT 0, 1000
210	04:19:10	COMMIT
211	04:19:37	SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ
212	04:20:02	SELECT precio FROM producto WHERE id_producto = 1 LIMIT 0, 1000
213	04:21:08	UPDATE producto SET precio = precio + 50 WHERE id_producto = 1

Primera imagen (1°A → 4°A)

- 1°A: hicimos el SELECT inicial para ver los precios y elegiste el producto con id_producto = 1.
- 2°A: configuramos el nivel de aislamiento READ COMMITTED y leímos el precio (1069.04).
- 3°B: otra sesión actualizó el precio sumando 50 y confirmó con COMMIT.
- 4°A: volvimos a leer el precio y vimos 1119.04, o sea, el cambio se reflejó dentro de la misma transacción.

Esto demuestra que en READ COMMITTED la transacción puede ver los cambios hechos por otra sesión (lectura no repetible).

Segunda imagen (5°A → 7°A)

- 5°A: configuramos el nivel REPEATABLE READ y leímos el precio (1119.04).
- 6°B: otra sesión volvió a actualizar el mismo registro (+50) y confirmó con COMMIT.
- 7°A: repetimos el SELECT dentro de la misma transacción y el precio siguió siendo 1119.04, sin reflejar el cambio.

Esto demuestra que en REPEATABLE READ la transacción mantiene la “foto” inicial y no ve los cambios externos hasta que hace COMMIT.

Figura 17. Comparación de niveles de aislamiento (READ COMMITTED vs REPEATABLE READ)

Se realizaron pruebas en dos sesiones concurrentes sobre la tabla `producto`, modificando el precio de un mismo registro bajo distintos niveles de aislamiento.

En READ COMMITTED, la transacción pudo ver el nuevo valor actualizado por otra sesión dentro de la misma transacción (lectura no repetible).

En REPEATABLE READ, la transacción mantuvo el valor original en todas las lecturas, aun cuando otra sesión modificó el registro, evidenciando la consistencia de lectura garantizada por este nivel de aislamiento.

Este experimento demuestra cómo el nivel de aislamiento afecta la visibilidad de los cambios y la estabilidad de los datos en entornos concurrentes.

Conclusiones

El TFI permitió integrar diseño, implementación, carga masiva, consultas avanzadas, seguridad y concurrencia en un entorno realista.

El sistema resultante es robusto, consistente y alineado con las buenas prácticas profesionales.

La IA se utilizó como tutoría pedagógica, apoyando el razonamiento y la validación sin reemplazar el trabajo crítico del equipo.

Anexo IA

A continuación se pegarán algunas de las preguntas que se hicieron a la IA, se utilizó Copilot y ChatGPT:

Estoy armando el DER y no sé si conviene usar el mail como PK o un id numérico. ¿Qué sería mejor a nivel diseño?

Esta es **la típica duda de diseño** que todos tenemos al empezar un DER, y es una excelente pregunta porque define la calidad del modelo a largo plazo.

Voy directo al punto, con claridad total:

★ Respuesta corta

Conviene usar **un id numérico autoincremental como PK** y dejar el mail como **UNIQUE**.

★ Respuesta explicada:

Mirá, usar el mail como clave primaria *parece tentador* porque es único...

pero en diseño profesional **no es una buena práctica**.

✓ 1. El mail NO es inmutable

Las personas cambian de correo:

- cambian de proveedor,
- pierden acceso,
- se casan y cambian apellido,
- quieren usar otro correo para la app.

Si tu PK cambia, tenés que actualizar **todas las FKs** que lo referencian.

Eso es carísimo y riesgoso.

✓ 2. Una PK debe ser simple, corta y sin significado

El mail:

- es largo,
- tiene semántica,
- depende del usuario,
- no es eficiente como índice.

Una PK ideal es:

- numérica,
- corta,
- sin significado,
- autogenerada.

✓ 3. Performance

Un **INT** como PK:

- indexa más rápido,
- ocupa menos espacio,
- mejora JOINS,
- mejora FKs.

Un **VARCHAR(100)** como PK:

- es más lento,
- ocupa más memoria,
- hace más costosos los JOIN.

✓ 4. El mail igual puede ser único

Lo correcto es:

- **id_usuario** → **PRIMARY KEY**
- **mail** → **UNIQUE**

Así garantizas unicidad sin comprometer el diseño.

★ Conclusión

La mejor práctica es:

sql

id_usuario INT AUTO_INCREMENT PRIMARY KEY,

mail VARCHAR(100) UNIQUE

Esto te da:

- PK estable,
- mail único,
- modelo profesional,

- consultas más rápidas.

¿Cómo agrego un CHECK para asegurar que el stock nunca sea negativo?

Esta es de esas preguntas cortitas pero clave para que el modelo quede **robusto y profesional**.

★ Respuesta clara y directa

Para asegurar que el stock nunca sea negativo, agregás un **CHECK** en la definición de la tabla:

sql

```
stock INT NOT NULL CHECK (stock >= 0)
```

★ Explicación

El **CHECK** es una restricción de integridad de dominio:

le dice a la base de datos *“solo aceptá valores dentro de este rango”*.

En este caso:

- **stock >= 0**
- evita valores negativos
- protege la consistencia del inventario
- y te permite detectar errores de carga o lógica antes de que se propaguen

Si intentás insertar:

sql

```
INSERT INTO producto (nombre, stock) VALUES ('X', -5);
```

MariaDB/MySQL te devuelve un error del tipo:

Error Code: 4025 – CHECK constraint failed

Eso es exactamente lo que querés demostrar en la Etapa 1.

★ Dónde colocarlo

Dentro del `CREATE TABLE producto`, por ejemplo:

sql

```
CREATE TABLE producto (  
    id_producto INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    descripcion VARCHAR(200),  
    precio DECIMAL(10,2) NOT NULL,  
    stock INT NOT NULL CHECK (stock >= 0),  
    eliminado TINYINT NOT NULL DEFAULT 0,  
    createdAt DATETIME NOT NULL DEFAULT NOW(),  
    id_categoria INT NOT NULL,  
    FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria)  
);
```

★ Conclusión para el TFI

Este CHECK cumple con la regla de negocio:

“El stock de los productos debe ser mayor o igual a cero en todo momento.”

Y además te permite mostrar una inserción inválida como parte de las pruebas de integridad.

¿Cómo puedo generar miles de registros sin escribirlos uno por uno?
¿Hay alguna técnica simple con INSERT...SELECT?

¡Esta es una de las mejores preguntas que podés hacer!

Porque justamente **nadie** va a escribir miles de registros a mano, y la cátedra espera que uses técnicas de SQL puro como INSERT...SELECT, JOIN y RAND().

Te respondo como si estuviéramos en pleno proceso del TFI, guiándote paso a paso.

★ Respuesta clara y directa

Sí, podés generar miles de registros usando **INSERT...SELECT**, combinando tablas semilla, RAND() y JOINS.

Es la técnica más simple, más limpia y más usada en la vida real.

★ Explicación guiada (como tutoría de IA)

✓ 1. Creás una tabla semilla o usás una existente

Por ejemplo, si ya tenés 200 usuarios, podés generar pedidos multiplicando esos usuarios.

✓ 2. Usás un SELECT que genere combinaciones

Ejemplo simple:

sql

```
INSERT INTO pedido (id_usuario, fecha, forma_pago, estado, eliminado, total, createdAt)
```

```
SELECT
```

```
  u.id_usuario,
```

```
  NOW() - INTERVAL FLOOR(RAND() * 30) DAY,
```

```
  ELT(FLOOR(1 + RAND()*3), 'EFFECTIVO', 'TARJETA', 'TRANSFERENCIA'),
```

```
  ELT(FLOOR(1 + RAND()*3), 'PENDIENTE', 'CONFIRMADO', 'TERMINADO'),
```

```
  0,
```

```
  0,
```

```
  NOW()
```

```
FROM usuario u
```

```
JOIN (
```

```
  SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
```

```
) r
```

```
ON RAND() < 0.3;
```

✓ ¿Qué hace esto?

- Toma cada usuario
- Lo combina con 1–4 filas artificiales
- Y solo inserta algunos (por el `RAND() < 0.3`)
- Resultado: **entre 5 y 20 pedidos por usuario**

Con 200 usuarios → generás entre **1000 y 4000 pedidos** sin escribir nada a mano.

★ 3. Para productos, podés hacer algo parecido

sql

```
INSERT INTO producto (nombre, descripcion, precio, stock, eliminado, createdAt, id_categoria)
```

```
SELECT
```

```
  CONCAT('Producto ', n),
```

```
  'Descripción generada',
```

```
  ROUND(RAND()*2000, 2),
```

```
  FLOOR(RAND()*100),
```

```
  0,
```

```
  NOW(),
```

```
  FLOOR(1 + RAND()*8)
```

```
FROM (
```

```
  SELECT @row := @row + 1 AS n
```

```
  FROM (SELECT 0 UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4) a,
```

```
       (SELECT 0 UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4) b,
```

```
       (SELECT @row := 0) r
```

```
) x;
```

Esto genera **25 productos**, pero si duplicás los bloques **a** y **b**, podés generar **200, 500, 2000 o 10.000**.

★ 4. ¿Por qué INSERT...SELECT es tan útil?

Porque te permite:

- generar miles de filas sin loops
- usar funciones aleatorias
- respetar FKs
- controlar distribuciones
- evitar escribir datos a mano

Es exactamente lo que la cátedra espera.

¿Cómo hago para generar fechas aleatorias dentro de los últimos 30 días?

¡Esta es una de las preguntas más típicas!

Y la respuesta es súper simple cuando entendés la lógica de las funciones de fecha en SQL.

Te respondo guiándote paso a paso.

★ Respuesta clara y directa

Podés generar fechas aleatorias dentro de los últimos 30 días usando:

sql

```
NOW() - INTERVAL FLOOR(RAND() * 30) DAY
```

★ Explicación guiada (como tutoría de IA)

✓ ¿Qué hace `RAND()`?

Genera un número decimal entre 0 y 1.

✓ ¿Qué hace `RAND() * 30`?

Lo convierte en un número entre 0 y 30.

Ejemplos:

- $0.12 \rightarrow 3.6$
- $0.87 \rightarrow 26.1$
- $0.01 \rightarrow 0.3$

✓ ¿Qué hace `FLOOR()`?

Redondea hacia abajo para obtener un entero:

- $3.6 \rightarrow 3$
- $26.1 \rightarrow 26$
- $0.3 \rightarrow 0$

✓ ¿Qué hace `NOW() - INTERVAL X DAY`?

Resta X días a la fecha actual.

★ Resultado final

Cada fila generada recibe una fecha aleatoria entre:

- **hoy**
- **hace 30 días**

Perfecto para pedidos, registros, logs, etc.

★ Ejemplo dentro de un INSERT masivo

sql

```
INSERT INTO pedido (id_usuario, fecha, forma_pago, estado, eliminado, total, createdAt)
```

```
SELECT
```

```
  u.id_usuario,
```

```
  NOW() - INTERVAL FLOOR(RAND() * 30) DAY,
```

```
  ELT(FLOOR(1 + RAND()*3), 'EFFECTIVO', 'TARJETA', 'TRANSFERENCIA'),
```

```
  ELT(FLOOR(1 + RAND()*3), 'PENDIENTE', 'CONFIRMADO', 'TERMINADO'),
```

```
  0,
```

```
  0,
```

```
  NOW()
```

```
FROM usuario u;
```

Esto te genera pedidos con fechas distribuidas de forma natural en el último mes.

¿Cómo verifico que no se generaron FK huérfanas después de la carga masiva?

¡Esta es EXACTAMENTE la pregunta que un estudiante haría en la Etapa 2, Maga!

Y la respuesta es súper simple, súper útil y totalmente alineada con lo que hiciste en tu TPI.

Te respondo como si estuviéramos trabajando juntas en la carga masiva.

★ Respuesta clara y directa

Para verificar que no se generaron FK huérfanas, hacés un **LEFT JOIN** desde la tabla hija hacia la tabla padre y filtrás los casos donde la PK de la tabla padre sea **NULL**.

Si el resultado es **0**, no hay huérfanas.

★ Ejemplo para detalle_pedido → pedido

sql

```
SELECT COUNT(*) AS detalles_sin_pedido
FROM detalle_pedido d
LEFT JOIN pedido p ON d.id_pedido = p.id_pedido
WHERE p.id_pedido IS NULL;
```

Si devuelve **0**, está perfecto.

★ Ejemplo para detalle_pedido → producto

sql

```
SELECT COUNT(*) AS detalles_sin_producto
FROM detalle_pedido d
LEFT JOIN producto pr ON d.id_producto = pr.id_producto
WHERE pr.id_producto IS NULL;
```

★ Ejemplo para producto → categoria

sql

```
SELECT COUNT(*) AS productos_con_categoria_invalida
```

```
FROM producto p
```

```
LEFT JOIN categoria c ON p.id_categoria = c.id_categoria
```

```
WHERE c.id_categoria IS NULL;
```

★ Explicación guiada (como tutoría de IA)

- El **LEFT JOIN** trae todos los registros de la tabla hija.
- Si la FK apunta a algo inexistente, la tabla padre devuelve **NULL**.
- Filtrando por **IS NULL** encontrarás los errores.
- Si el conteo es **0**, tu carga masiva está **consistente**.

Este es el método estándar que usan los profes, los DBAs y los desarrolladores.

Estas consultas son perfectas para incluir como **pruebas de consistencia** en la Etapa 2.

No logro hacer un JOIN múltiple entre pedido, usuario y detalle_pedido para obtener el total del pedido

Es normal trabarse acá porque un JOIN múltiple parece complicado... hasta que entendés la lógica.

Te respondo como si estuviéramos tomando mate.

★ Respuesta clara y directa

Para obtener el total del pedido, necesitás unir:

- **pedido** → tabla principal
- **usuario** → quién hizo el pedido
- **detalle_pedido** → líneas del pedido

La estructura correcta del JOIN es:

sql

```
SELECT
```

```
    p.id_pedido,
```



```
u.nombre,  
  
u.apellido,  
  
SUM(d.subtotal) AS total_pedido  
  
FROM pedido p  
  
JOIN usuario u ON p.id_usuario = u.id_usuario  
  
JOIN detalle_pedido d ON p.id_pedido = d.id_pedido  
  
GROUP BY p.id_pedido;
```

★ Explicación guiada (como tutoría de IA)

✓ 1. Empezás por la tabla principal

pedido es la tabla que “manda” en esta consulta.

sql

```
FROM pedido p
```

✓ 2. Unís usuario por su FK

Cada pedido tiene un usuario asociado.

sql

```
JOIN usuario u ON p.id_usuario = u.id_usuario
```

✓ 3. Unís detalle_pedido por su FK

Cada pedido tiene uno o varios detalles.

sql

```
JOIN detalle_pedido d ON p.id_pedido = d.id_pedido
```

✓ 4. Sumás los subtotales

Cada detalle tiene un subtotal → sumarlos te da el total del pedido.

sql

SUM(d.subtotal)

✓ 5. Agrupás por pedido

Porque un pedido tiene varios detalles.

sql

GROUP BY p.id_pedido

★ Errores comunes que explican por qué no te salía

✗ 1. Hacer el JOIN en el orden incorrecto

Si empezás por detalle_pedido, te duplica filas.

✗ 2. Olvidar el GROUP BY

Sin GROUP BY, MySQL te devuelve error o resultados incoherentes.

✗ 3. Usar WHERE en vez de JOIN

El WHERE no sirve para unir tablas, solo para filtrar.

✗ 4. No tener subtotales calculados

Si **subtotal** está en NULL, SUM devuelve NULL

¿Me explicás el EXPLAIN? No entiendo qué significa que el tipo sea “ALL” o “ref”.

Esta es **LA** pregunta clave para entender optimización en SQL.

Y te juro que cuando entendés EXPLAIN, te cambia la cabeza para siempre.

★ Respuesta clara y directa

En EXPLAIN, la columna **type** indica *cómo* MySQL/MariaDB accede a la tabla.

Es una medida de **eficiencia**.

Los valores más comunes son:

- **ALL** → *peor caso*: full table scan
- **ref** → usa un índice para buscar coincidencias
- **eq_ref** → *mejor caso* en JOINS: coincidencia exacta por PK/FK

★ Explicación guiada (como tutoría de IA)

● 1. type = ALL

Significa:

“Voy a leer **todas** las filas de la tabla.”

Es el peor escenario porque:

- no usa índice
- escanea toda la tabla
- es lento con miles de registros

Ejemplo típico:

- cuando no hay índice en la columna del JOIN
- cuando hacés un WHERE sin índice
- cuando la tabla hija no tiene FK indexada

En tu EXPLAIN, lo viste en:

Código

table: d

type: ALL

Eso significa que **detalle_pedido** no tenía índice en **id_pedido** (o no lo estaba usando).

● 2. type = ref

Significa:

“Voy a usar un índice para buscar filas que coincidan con un valor.”

Es mucho más eficiente que ALL.

Se usa cuando:

- hay un índice en la FK
- hay un índice en la columna del WHERE
- hay un índice en la columna del JOIN

Ejemplo típico:

Código

table: producto

type: ref

key: id_categoria

Esto es bueno: MySQL usa el índice para encontrar productos por categoría.

● 3. type = eq_ref

Este es **el mejor tipo de acceso posible** en un JOIN.

Significa:

“Por cada fila de la tabla principal, voy a buscar **exactamente una** fila en la tabla unida, usando una PK o UNIQUE.”

Es perfecto para:

- JOIN por PK
- JOIN por FK con índice
- relaciones 1:1 o 1:N bien indexadas

En tu EXPLAIN lo viste así:

Código

table: u

type: eq_ref

key: PRIMARY

Eso significa:

- MySQL está usando la PK de usuario

- El JOIN está optimizado
- No hay duplicaciones ni búsquedas innecesarias

★ Resumen rápido para tu TFI

type	Significado	Rendimiento
ALL	Full table scan	✗ Malo
ref	Usa índice para buscar coincidencias	✓ Bueno
eq_ref	Coincidencia exacta por PK/FK	★ Excelente

★ Cómo lo explicaría un docente

- **ALL** = MySQL lee toda la tabla → lento
- **ref** = MySQL usa un índice → rápido
- **eq_ref** = MySQL usa PK/FK → óptimo

★ Conclusión para tu informe

Podés escribir algo así:

“El EXPLAIN mostró que la tabla detalle_pedido se accedía con type = ALL, indicando un full table scan. En cambio, usuario y pedido se accedieron con eq_ref, lo que confirma el uso de claves primarias y foráneas correctamente indexadas.”

.

¿Cómo creo un usuario con privilegios mínimos para que solo pueda ver vistas y crear pedidos?

Te respondo guiándote paso a paso.

★ Respuesta clara y directa

Para crear un usuario con privilegios mínimos que solo pueda:

- ver **vistas públicas**, y
- insertar **pedidos**,

tenés que:

1. Crear el usuario
2. Revocarle todo
3. Darle solo SELECT sobre vistas
4. Darle solo INSERT sobre pedido

★ Explicación guiada (como tutoría de IA)

- ✓ 1. Crear el usuario

sql

```
CREATE USER 'app_user'@'localhost' IDENTIFIED BY '1234';
```

- ✓ 2. Revocar cualquier permiso previo

Esto es buena práctica para asegurarte de que arranca “limpio”.

sql

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'app_user'@'localhost';
```

- ✓ 3. Darle SELECT solo sobre las vistas públicas

Esto evita que vea tablas internas con datos sensibles.

sql

```
GRANT SELECT ON tpi_bd.vista_usuarios_publicos TO 'app_user'@'localhost';
```

```
GRANT SELECT ON tpi_bd.vista_pedidos_publicos TO 'app_user'@'localhost';
```

- ✓ 4. Darle permiso para insertar pedidos

Esto simula el comportamiento típico de una app real.

sql

```
GRANT INSERT ON tpi_bd.pedido TO 'app_user'@'localhost';
```

✓ 5. Aplicar cambios

```
sql
```

```
FLUSH PRIVILEGES;
```

★ ¿Por qué esto es “privilegios mínimos”?

Porque el usuario:

- ✗ NO puede ver tablas internas
- ✗ NO puede borrar
- ✗ NO puede actualizar
- ✗ NO puede ver contraseñas ni datos sensibles
- ✓ Solo puede ver vistas filtradas
- ✓ Solo puede crear pedidos

Esto cumple exactamente con el principio de **least privilege** que pide la cátedra.

★ Errores típicos que explican por qué a veces no funciona

✗ 1. Intentar dar permisos sobre vistas que todavía no existen

Esto genera el error 1146.

✗ 2. Crear la base de datos dos veces

Error 1007.

✗ 3. Dar permisos sobre tablas en vez de vistas

Eso rompe el objetivo de seguridad.

No sé cómo hacer un procedimiento almacenado que sea seguro y no permita SQL injection

Esta es **LA** pregunta clave.

Y te juro que si entendés esto, ya entendiste el 90% de cómo evitar SQL injection en SQL puro.

Te respondo como si estuviéramos trabajando juntas en el TFI, guiándote paso a paso.

★ Respuesta clara y directa

Un procedimiento almacenado es seguro **si NO usa SQL dinámico y NO concatena cadenas**.

En vez de eso, usa **parámetros** que el motor valida internamente.

★ Ejemplo de procedimiento seguro (sin SQL dinámico)

sql

```
CREATE PROCEDURE buscar_usuario_seguro(IN p_mail VARCHAR(100))
```

```
BEGIN
```

```
    SELECT id_usuario, nombre, apellido, mail
```

```
    FROM usuario
```

```
    WHERE mail = p_mail;
```

```
END;
```

★ Por qué esto es seguro

Porque:

- **p_mail** es un **parámetro**, no una cadena concatenada
- MySQL **no interpreta** el contenido como código
- Si alguien intenta inyectar ' **OR 1=1 --**, el motor lo trata como **texto literal**
- No hay **CONCAT()**, ni **PREPARE**, ni **EXECUTE** → por lo tanto **no hay SQL dinámico**

Esto bloquea automáticamente cualquier intento de inyección.

★ Ejemplo de ataque que queda neutralizado

Si alguien intenta:

Código

```
CALL buscar_usuario_seguro("'" OR 1=1 --");
```


El motor lo interpreta como:

Código

WHERE mail = "" OR 1=1 --"

→ **No devuelve nada**

→ **No rompe la consulta**

→ **No ejecuta código malicioso**

Eso es exactamente lo que la cátedra quiere ver.

★ Cómo NO debe hacerse (inseguro)

Esto sería inseguro:

sql

```
SET @sql = CONCAT('SELECT * FROM usuario WHERE mail = ', p_mail, '');
```

```
PREPARE stmt FROM @sql;
```

```
EXECUTE stmt;
```

¿Por qué?

Porque acá **sí** se ejecuta lo que el usuario escribe.

Y si el usuario mete código malicioso, se ejecuta.

★ Conclusión para tu TFI

Un procedimiento seguro:

- usa parámetros
- no concatena strings
- no usa SQL dinámico
- no usa PREPARE/EXECUTE
- no interpreta el input como código

Estoy intentando simular un deadlock pero no aparece el error 1213.
No sé que estoy haciendo mal

Maga, esta pregunta es **exactamente la que aparece siempre** cuando alguien intenta simular un deadlock por primera vez.

Y te juro que el 90% de las veces la razón es **la misma**:

👉 **estás ejecutando todo en una sola sesión.**

Te lo explico como si estuviéramos trabajando juntas en el TFI.

★ Respuesta clara y directa

Si no aparece el error 1213, es porque **no estás usando dos sesiones reales.**

MySQL/MariaDB **no puede generar un deadlock dentro de una misma conexión**, aunque vos escribas “Sesión 1” y “Sesión 2” en el script.

★ Explicación guiada (como tutoría de IA)

🔵 1. Un deadlock requiere dos conexiones independientes

Cada conexión mantiene sus propios bloqueos.

Si todo se ejecuta en la misma pestaña:

- MySQL no compite consigo mismo
- no hay dos transacciones activas
- no hay bloqueo cruzado
- **no puede haber deadlock**

Por eso no aparece el error 1213.

🔵 2. Cómo saber si estás en una sola sesión

Si estás usando:

- una sola pestaña de Workbench
- una sola consola
- un solo script ejecutado de arriba a abajo

→ **es una sola sesión**, aunque vos lo separes en bloques.

3. Cómo simular un deadlock REAL

Tenés que abrir **dos pestañas distintas** en Workbench:

✓ Sesión 1:

sql

START TRANSACTION;

UPDATE pedido SET estado = 'PENDIENTE' WHERE id_pedido = 1;

(No hagas COMMIT todavía)

✓ Sesión 2:

sql

START TRANSACTION;

UPDATE detalle_pedido SET cantidad = cantidad + 1 WHERE id_pedido = 1;

(No hagas COMMIT todavía)

✓ Volvés a Sesión 1:

sql

UPDATE detalle_pedido SET cantidad = cantidad + 1 WHERE id_pedido = 1;

✓ Volvés a Sesión 2:

sql

UPDATE pedido SET estado = 'CONFIRMADO' WHERE id_pedido = 1;

 **Acá aparece el error 1213**

Porque:

- Sesión 1 bloqueó **pedido**
- Sesión 2 bloqueó **detalle_pedido**

- Ambas quieren lo que tiene la otra
- Ninguna puede avanzar
- MySQL detecta el deadlock y mata una transacción

★ 4. Señales de que el deadlock está funcionando

Cuando lo hacés bien:

- una sesión queda “esperando”
- la otra devuelve el error 1213

aparece el mensaje:

- **Deadlock found when trying to get lock; try restarting transaction**

¿MariaDB no reconoce la variable `transaction_isolation`?

Esta pregunta es **exactamente la que aparece siempre** cuando uno trabaja con MariaDB y sigue ejemplos de MySQL 8.

Y la respuesta es simple: **no estás haciendo nada mal**.

MariaDB directamente **no usa** la variable `transaction_isolation`.

Te lo explico como si estuviéramos trabajando juntas en la Etapa 5.

★ Respuesta clara y directa

MariaDB **no reconoce** `@@transaction_isolation` porque **esa variable existe solo en MySQL 8**.

En MariaDB, la variable equivalente se llama:

👉 `tx_isolation`

★ Explicación guiada (como tutoría de IA)

1. MySQL y MariaDB NO usan los mismos nombres de variables

MySQL 8 usa:

```
sql
```

```
SELECT @@transaction_isolation;
```

MariaDB usa:

sql

```
SELECT @@tx_isolation;
```

Por eso cuando probaste:

sql

```
SELECT @@transaction_isolation;
```

te devolvió:

Error Code: 1193 – Unknown system variable 'transaction_isolation'

No es tu culpa.

Es simplemente que MariaDB tiene otra sintaxis.

2. Cómo ver el nivel de aislamiento en MariaDB

Tenés dos opciones válidas:

✓ Sesión actual:

sql

```
SELECT @@session.tx_isolation;
```

✓ Global:

sql

```
SELECT @@global.tx_isolation;
```

3. Cómo cambiar el nivel de aislamiento en MariaDB

También cambia la sintaxis respecto a MySQL.

✗ Esto NO funciona en MariaDB:

sql

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

✓ Esto SÍ funciona:

sql

```
SET SESSION tx_isolation = 'READ-COMMITTED';
```

4. Valores válidos en MariaDB

Son exactamente estos:

- 'READ-UNCOMMITTED'
- 'READ-COMMITTED'
- 'REPEATABLE-READ'
- 'SERIALIZABLE'

Y por defecto, MariaDB usa:

👉 **REPEATABLE-READ**

¿Qué diferencia hay entre READ COMMITTED y REPEATABLE READ?

Esta es **LA** pregunta clave de niveles de aislamiento.

Si entendés esta diferencia, entendés el corazón de la Etapa 5.

Te lo explico con claridad total y ejemplos simples.

★ Respuesta clara y directa

La diferencia principal es:

READ COMMITTED → cada SELECT ve **solo datos ya confirmados** (committed).

- Si otra transacción hace COMMIT, vos **sí** ves los cambios.

REPEATABLE READ → cada SELECT ve **siempre la misma foto** de los datos durante toda la transacción.

- Aunque otra transacción haga COMMIT, vos **no** ves los cambios hasta terminar.

★ Explicación guiada (como tutoría de IA)

● READ COMMITTED (lectura confirmada)

Es más flexible, más “realista”.

✓ Qué permite:

- Ver cambios confirmados por otras transacciones mientras la tuya sigue abierta.

✓ Qué evita:

- Lecturas sucias (dirty reads).

✗ Qué NO evita:

Lecturas no repetibles (non-repeatable reads).

- Es decir: **si lees dos veces, podés obtener valores distintos.**

🔍 Ejemplo:

Sesión A lee el stock → 10

Sesión B actualiza stock a 5 y hace COMMIT

Sesión A vuelve a leer → ahora ve 5

👉 **La lectura cambió.**

● REPEATABLE READ (lectura repetible)

Es más estricto, más seguro.

✓ Qué garantiza:

- Si lees un dato una vez, **lo verás igual durante toda la transacción.**
- Aunque otra transacción haga COMMIT, vos seguís viendo la “foto inicial”.

✓ Qué evita:

- Lecturas sucias
- Lecturas no repetibles

✗ Qué NO evita (en MySQL/MariaDB):

- Lecturas fantasma (aunque InnoDB usa next-key locks y suele evitarlas también)

🔍 Ejemplo:

Sesión A lee stock → 10

Sesión B actualiza stock a 5 y hace COMMIT

Sesión A vuelve a leer → **sigue viendo 10**

👉 La lectura se mantiene estable.

★ Resumen rápido

Nivel	¿Ve cambios de otras transacciones durante la transacción?	¿Evita lecturas no repetibles?
READ COMMITTED	Sí	✗ No
REPEATABLE READ	No	✓ Sí